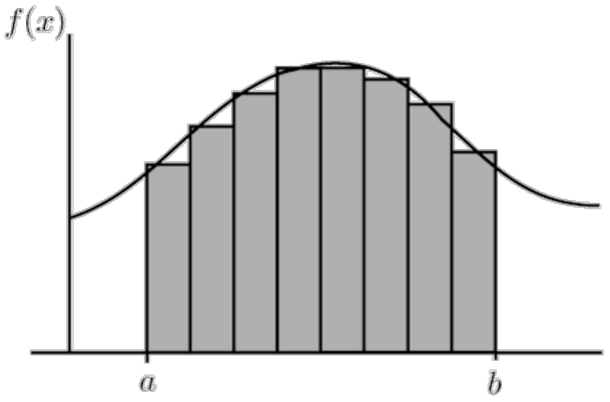
Riemann's Integral in Numerical Methods

- **Definition**: Riemann's Integral is a method of approximating integrals by summing areas of rectangles.

- **Approach**:
  - **Rectangle Width (h)**: The width of each rectangle is a uniform distance across subintervals.
  - **Rectangle Height**: Calculated based on the function's value at specific points (e.g., left or right endpoints of the interval).

  - **Formula**:

$$\int_a^b f(x)dx \approx \sum_{i=0}^{n-1} hf(x_i), \qquad \text{or} \qquad \int_a^b f(x)dx \approx \sum_{i=1}^{n} hf(x_i),$$



Choice of x_i : Left (x_i) or Right (x_{i+1}) endpoint.

- **Goal**: Achieve higher accuracy by reducing h (i.e., making subintervals finer).

Mathematical Foundation & Error Analysis

- **Taylor Series Expansion**:
  - Used to approximate the function f(x) over subintervals.
  - Taylor expansion:

$$f(x) = f(x_i) + f'(x_i)(x - x_i) + \cdots$$

$$\int_{x_i}^{x_{i+1}} f(x)dx = \int_{x_i}^{x_{i+1}} (f(x_i) + f'(x_i)(x - x_i) + \cdots)\, dx$$

- Integral over subinterval approximated as:

$$\int_{x_i}^{x_{i+1}} f(x)dx = hf(x_i) + \frac{h^2}{2} f'(x_i) + O(h^3),$$

$$\int_{x_i}^{x_{i+1}} f(x)dx = hf(x_i) + O(h^2).$$

- **Error Calculation**:
  - **Single Interval**: Approximation error is O(h^2) .
  - **Whole Interval**: Summed error across intervals yields O(h) , making finer subintervals crucial for accuracy.

Midpoint Rule in Numerical Integration

- **Definition**: The Midpoint Rule is an improved technique for approximating integrals by taking the function value at the midpoint of each subinterval.
- **Approach**:
  - **Midpoint Calculation**:
    - For each interval [x_i, x_{i+1}] , the midpoint is y_i = (x_i + x_{I+1}/2 .
    - The function value f(y_i) at the midpoint provides the rectangle's height.
  - **Formula**:

$$\int_a^b f(x)dx \approx \sum_{i=0}^{n-1} hf(y_i).$$

where h is the width of each subinterval.

- **Advantages**:
  - **Higher Accuracy**: The Midpoint Rule typically offers an error of O(h^2) , an improvement over the standard Riemann sum methods which are O(h) .
  - **Symmetry Benefit**: The symmetric position of the midpoint helps cancel out lower-order error terms in the Taylor expansion, leading to a more accurate result.
- **Limitations**:
  - **Data Dependency**: Requires function values at midpoints; if data points are fixed (not adjustable), midpoint values might not be computable.

Let's use the left Riemann Integral, right Riemann Integral, and Midpoint Rule to approximate $\int \sin(x)dx$ with 11 evenly spaced grid points over the whole interval. Compare to the exact value of 2.

```
import numpy as np

a = 0
b = np.pi
n = 11
h = (b - a) / (n - 1)
x = np.linspace(a, b, n)
f = np.sin(x)

I_riemannL = h * sum(f[:n-1])
err_riemannL = 2 - I_riemannL

I_riemannR = h * sum(f[1::])
err_riemannR = 2 - I_riemannR

I_mid = h * sum(np.sin((x[:n-1] \
        + x[1:])/2))
err_mid = 2 - I_mid

print(I_riemannL)
print(err_riemannL)

print(I_riemannR)
print(err_riemannR)

print(I_mid)
print(err_mid)
```

```
1.9835235375094546
0.01647646249054535
1.9835235375094546
0.01647646249054535
2.0082484079079745
-0.008248407907974542
```

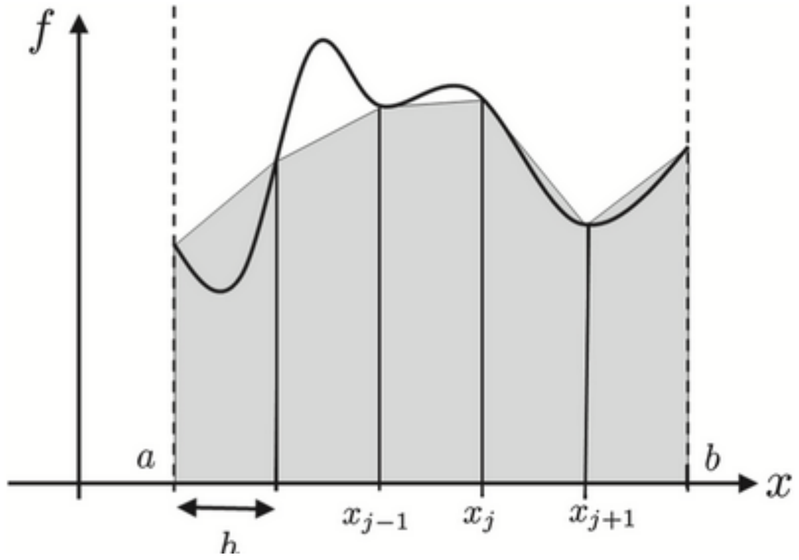
Trapezoid Rule for Numerical Integration

Concept Overview:

- The Trapezoid Rule approximates the integral of a function by dividing the area under the curve into trapezoidal segments.
- For each interval [x_i, x_{i+1}], the trapezoid is formed between the points (x_i, f(x_i)) and (x_{i+1}, f(x_{i+1})), making the area computation straightforward.
- Formula: The area of each trapezoid is calculated as:

$$\text{Area} = \frac{h}{2}(f(x_i) + f(x_{i+1}))$$

$$\int_a^b f(x)dx \approx \sum_{i=0}^{n-1} h\frac{f(x_i) + f(x_{i+1})}{2}.$$



where h is the width of each interval.

- Common in engineering and scientific applications where discrete data points approximate continuous functions.

Computational Efficiency and Practical Implementation

Efficient Computation:

- The Trapezoid Rule double-counts terms across intervals except at the boundaries, which can be computationally intensive. This can be optimized using the expression:

$$\int_a^b f(x)dx \approx \frac{h}{2}\left(f(x_0) + 2\left(\sum_{i=1}^{n-1} f(x_i)\right) + f(x_n)\right).$$

Trapezoid Rule to approximate $\int \sin(x)dx$ *with* 11 evenly spaced grid points over the whole interval. Compare to the exact value of 2.

```
import numpy as np

a = 0
b = np.pi
n = 11
h = (b - a) / (n - 1)
x = np.linspace(a, b, n)
f = np.sin(x)

I_trap = (h/2)*(f[0] + \
          2 * sum(f[1:n-1]) + f[n-1])
err_trap = 2 - I_trap

print(I_trap)
print(err_trap)
```

```
1.9835235375094546
0.01647646249054535
```

Error Analysis and Accuracy Considerations

Taylor Series Expansion for Error Estimation:

- The accuracy of the Trapezoid Rule can be evaluated by expanding f(x) as a Taylor series around the midpoint. This reveals that the error per interval is proportional to O(h^2).
- When applied over multiple intervals, the cumulative error becomes O(h) for the entire integration range.

Practical Implication:

- The method is more accurate for functions that are nearly linear over each subinterval.
- For functions with higher curvature, the accuracy decreases, and methods like Simpson's Rule may provide better precision.

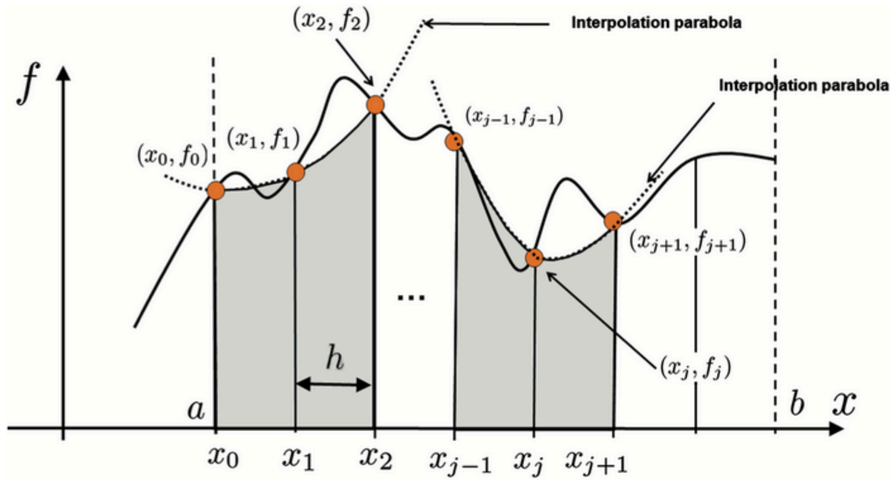
Simpson's Rule

- **Definition**: Simpson's Rule is a numerical method for approximating the integral of a function. It works by fitting a quadratic polynomial to segments of the function's curve and then calculating the area under this polynomial.
- **Concept**:
  - Uses **two consecutive subintervals** to approximate the integral over a range.
  - Applies a **parabolic approximation** through three key points (x_{i-1}, f(x_{i-1})), (x_i, f(x_i)), (x_{i+1}, f(x_{I+1})), using Lagrange polynomials.
  - Integrates this parabolic function exactly over each interval.
- **Formula**:

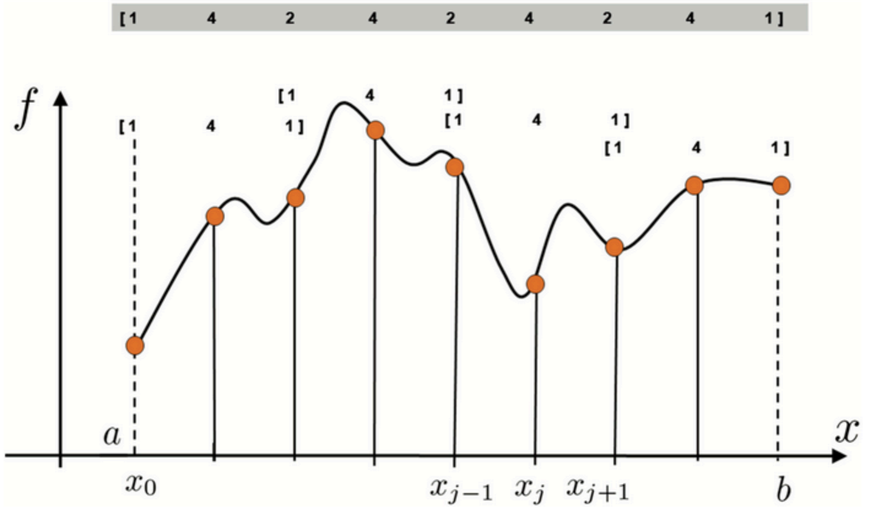
$$\int_a^b f(x)\,dx \approx \frac{h}{3}[f(x_0) + 4f(x_1) + 2f(x_2) + \ldots + 4f(x_{n-1}) + f(x_n)]$$

$$\int_a^b f(x)dx \approx \frac{h}{3}\left[f(x_0) + 4\left(\sum_{i=1,i\text{ odd}}^{n-1} f(x_i)\right) + 2\left(\sum_{i=2,i\text{ even}}^{n-2} f(x_i)\right) + f(x_n)\right]$$

where h is the width of each subinterval, requiring an **odd number of points**.



This regrouping is illustrated in the figure below:



SimpsonRule to approximate $\int \sin(x)dx$ with 11 evenly spaced grid points over the whole interval. Compare to the exact value of 2.

```
import numpy as np

a = 0
b = np.pi
n = 11
h = (b - a) / (n - 1)
x = np.linspace(a, b, n)
f = np.sin(x)

I_simp = (h/3) * (f[0] + 2*sum(f[:n-2:2]) \
            + 4*sum(f[1:n-1:2]) + f[n-1])
err_simp = 2 - I_simp

print(I_simp)
print(err_simp)
```

```
2.0001095173150043
-0.00010951731500430384
```

Advantages and Accuracy of Simpson’s Rule

- **Accuracy**:
  - Simpson’s Rule achieves **fourth-order accuracy**, making it more precise than the Trapezoid Rule.
  - It cancels certain error terms using the **Taylor Series expansion**, further enhancing accuracy over two intervals.

- **Considerations**:
  - Requires an **odd number of points** (even number of intervals).
  - Errors diminish rapidly with finer intervals but are highly effective for smooth functions with fewer divisions.

Numerical Integration in Python

- **Overview**: Numerical integration is essential for approximating definite integrals when analytic solutions are impractical. Python, with libraries like scipy.integrate, offers tools for implementing these methods efficiently.

- **Applications**: Common in engineering, physics, and computational science, numerical integration helps in scenarios involving irregular functions or data.

- **Key Methods**:
  - **Trapezoidal Rule**: Approximates the area under a curve using trapezoids.
  - **Cumulative Trapezoidal Integration**: Useful for analyzing incremental integration over intervals.
  - **Adaptive Quadrature Integration**: Higher accuracy method also providing an error estimate.

Let's use the *trapz* function to approximate $\int \sin(x)dx$ for 11 equally spaced points over the whole interval. Compare this value to the one computed in the earlier example using the Trapezoid Rule.

```
import numpy as np
from scipy.integrate import trapz

a = 0
b = np.pi
n = 11
h = (b - a) / (n - 1)
x = np.linspace(a, b, n)
f = np.sin(x)

I_trapz = trapz(f,x)
I_trap = (h/2)*(f[0] + 2 * sum(f[1:n-1]) + f[n-1])

print(I_trapz)
print(I_trap)
```

```
1.9835235375094542
1.9835235375094546
```

Sometimes we want to know the approximated cumulative integral. That is, we want to know $F(X) = \int_{x_0}^{X} f(x)dx$. For this purpose, it is useful to use the *cumtrapz* function *cumsum*, which takes the same input arguments as *trapz*.

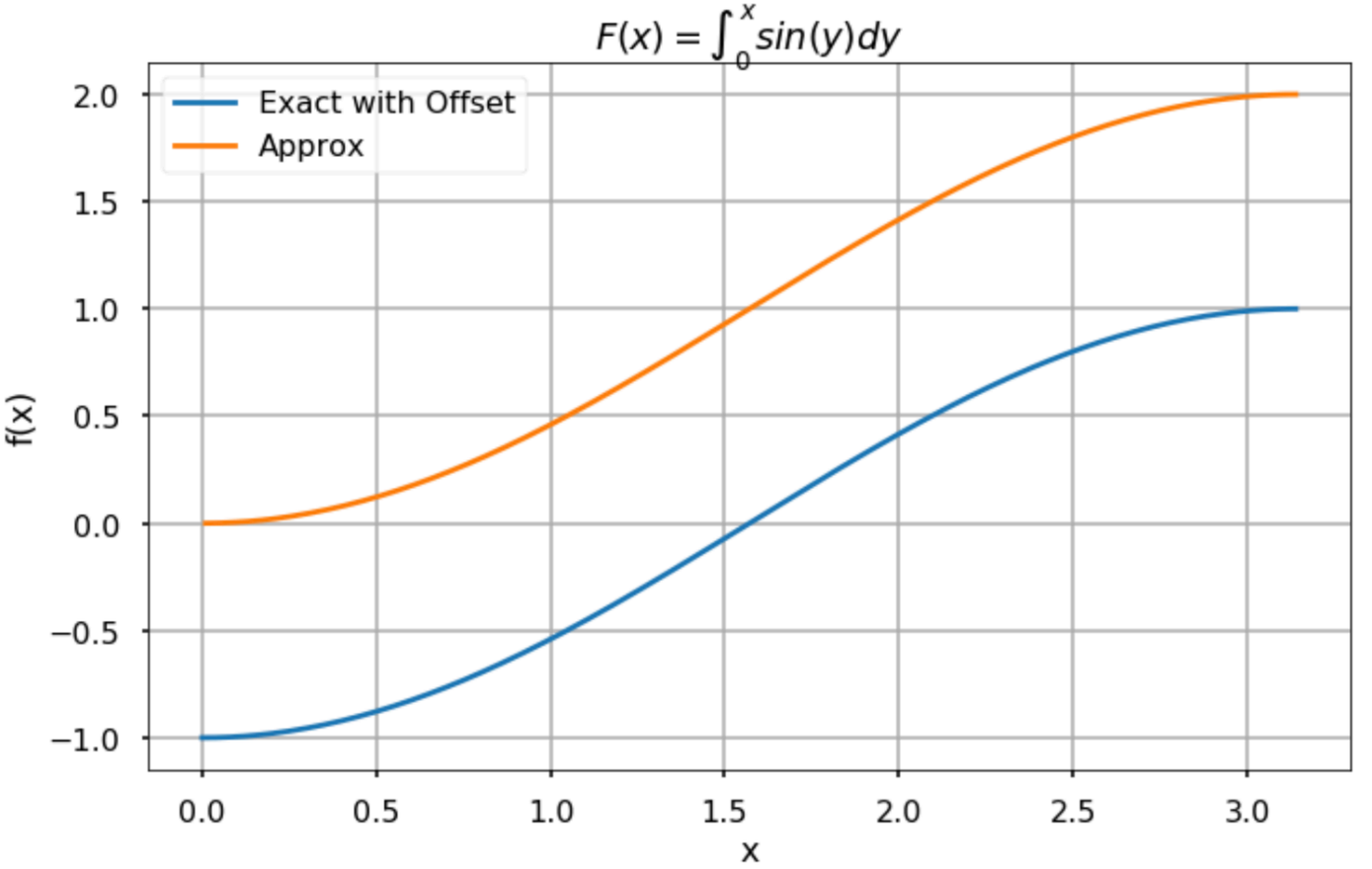
Let's use the *cumtrapz* function to approximate the cumulative integral of $f(x)$=sin(x) from 0 to π with a discretization step of 0.01. The exact solution of this integral is $F(x)$=-cos(x).

```
from scipy.integrate import cumtrapz
import matplotlib.pyplot as plt

%matplotlib inline
plt.style.use('seaborn-poster')

x = np.arange(0, np.pi, 0.01)
F_exact = -np.cos(x)
F_approx = cumtrapz(np.sin(x), x)

plt.figure(figsize = (10,6))
plt.plot(x, F_exact)
plt.plot(x[1::], F_approx)
plt.grid()
plt.tight_layout()
plt.title('$F(x) = \int_0^{x} sin(y) dy$')
plt.xlabel('x')
plt.ylabel('f(x)')
plt.legend(['Exact with Offset', 'Approx'])
plt.show()
```



Let's use the *integrate.quad* function to compute $\int \sin(x)dx$. Compare the answer with the correct answer of 2.

```
from scipy.integrate import quad

I_quad, est_err_quad = \
          quad(np.sin, 0, np.pi)
print(I_quad)
err_quad = 2 - I_quad
print(est_err_quad, err_quad)
```

```
2.0
2.220446049250313e-14 0.0
```